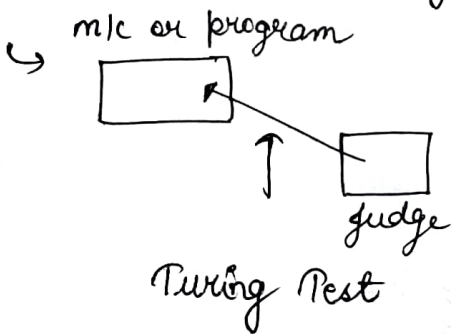


Artificial Intelligence

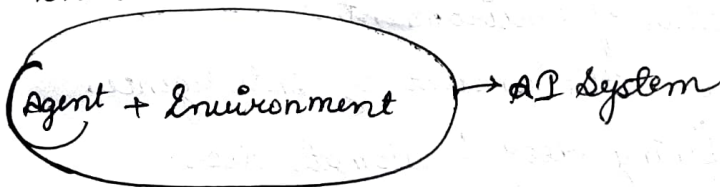
Definition

- ↳ study of intelligent agents (By Russell + Norvig)
- ↳ study of intelligent machines (by computer scientist John McCarthy in 1956)
- ↳ the term Art. Int. was coined in 1956 in Dartmouth Conference in 1956
- ↳ In 1950, Allen Turing raised a question "Can machines think"? → this gave base of AI



AI = Agents + Environment

- ↳ acc. to Russell + Norvig AI system is made from Agents and Environment.



- Agent is a combination of machine and program.
- Agent interacts with environment to execute a task or action
- In order to perform an action, agent percepts the environment to know its state and performs accordingly
- In order to build the state of environment, we / agent have to percept (take input from) the environment.
- agents percepts the environment via sensors and gets the

data i.e state of environment.

- Then agent takes the action (or decision). And the decision can change the state of environment.
- Action will be performed by actuator.

Then agent will perceive the changed ~~th~~ environment and then the process continues till the goal is reached.

Types of Agents & Environments

> Types of Agents -

- 1) Simple Reflex Agents
- 2) Model Based Reflex Agents
- 3) Goal Based Agents.
- 4) Utility Based Agents
- 5) Learning Agents.
- 6) Hybrid Agents.



increasing level
intelligence

Simple Reflex - It will perceive the environment and get the current status of environment.

- it has very little amount of intelligence.
eg - self-opening door / automatic door.
- no memory required
- rule based intelligent (if-else)

Model Based - It perceives the environment and creates the model related to that environment, and takes actions accordingly.

- some memory is needed in this case
- rule based + memory based

eg - room cleaning

Goal Based - • a goal is decided and decisions are taken to reach that goal.

eg- path planner using CV, chess player.

- agent takes actions based on certain plan
- ~~agent~~ must know state of environment, to execute the plan.
 results of past data / actions and consequences of actions

Utility Based - • agent passes a utility function that gives the cost of every action done to change a state from one to another.

- here goal focus is to reach the goal with optimum cost.

eg- cloud resource scheduling ^{agents}, stock trading agents.

Learning Agents - • ml agents or dl agents

- these agents have ability to learn based on data
- they take actions based on model made after learning
- eg- self driving car, ~~image~~ face detection.

Hybrid Agents - combination

Types of Environment -

set of actions - actions space
set of state - state space

1) Static and Dynamic Environment

2) Discrete and Continuous Environment

if we can
count no. of states
↳ finite no. of
states

if we cannot
count no. of states
↳ infinite no. of
states

↳ agent needs to
take finite no. of
actions

↳ agent needs to take
infinite no. of actions

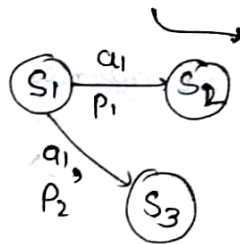
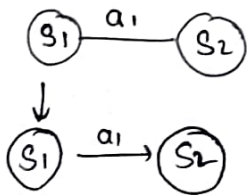
↳ modeled using
Markov Decision Process
(reinforcement learning)

↳ modeled with differential eqns
or non-linear continuous
functions.

↳ eg - env. of chess board.

↳ eg - env. of self driving
car.

3) Deterministic And Stochastic Environment



probabilistic reasoning is used
for modelling in a stochastic
env.

↳ simple rule
based reasoning is
enough to model in determ. env.

4) Fully Observable and Partially Observable

↳ if agent can
fully perceive the
env.

↳ agent can't fully observe the env.

↳ no memory is
req.

↳ memory is required to store the
outcome of action past actions.

↳ agent has full info.
of whole env.

↳ some info. is missing for about
env. for agent

↳ eg - puzzle

↳ eg - medical diagnostic
self driving car

6) Episodic And sequential Environment -

↳ where new event is indep. of previous event

↳ where new event is somewhat dep. on previous event

↳ current action is indep of past & future action.

↳ past action affects the current or future action.

Qualities of Agents to make an intelligent agent -

↳ capability of planning

↳ capability of learning

↳ decision making capability

↳ pattern recognition capability

↳ Problem solving skill

Fundamental Properties.

↳ beginning of AI.

Problem Solving

* in order to solve a problem, we must have to formulate the problem first.

* Some imp. personalities in CS
John McCarthy

Marvin Minsky

Geoffrey Hinton

Yann LeCun

Joshua Bengio

Stuart Russell

Andrew Ng (ML yearning)

Sam Altman - CEO of OpenAI

Elon Musk

Ilya Sutskever - student of Geoffrey Hinton

Jan Goodfellow - ^{gave} generative adversarial network concept

Demis Hassabis - CEO of google deepmind
_{buyed} → startup for AGI

D.H. Noble price winner in Chemistry

Problem Solving By Searching

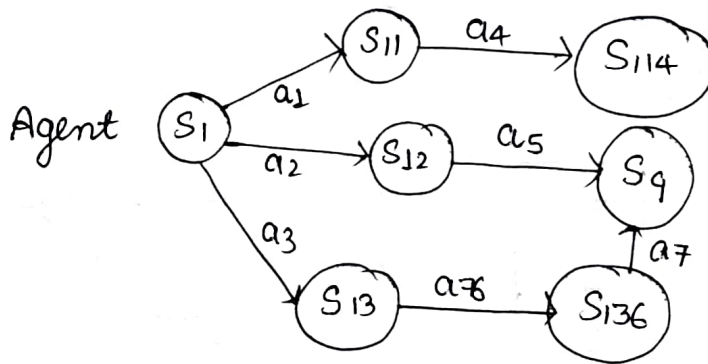
↳ problem is formulated by state & space search technique
↓
set of state

action space → set of possible actions

transition → $S_1 \xrightarrow{a} S_2$

action cost model → model associated with cost of actions of model

↳ In order to solve a problem, agent needs to reach from start state to last state (goal state)



State Space = $\{ S_1, S_{11}, S_{12}, S_{13}, S_{114}, S_{136}, S_9 \}$

Action Space = $\{ a_1, a_2, a_3, a_4, a_5, a_6, a_7 \}$

↳ In order to solve the problem, agent has to explore all the possible paths (brute force) this method is called exhaustiveness. property

↳ if the solutions, agent must reach the goal, this property is called completeness.

↳ optimality property.

* On the basis of above properties, there are three types of searches, namely

- ① Uninformed Search
- ② Informed Search
- ③ Adversarial Search

2	8	3
1	6	4
7		5

Initial State

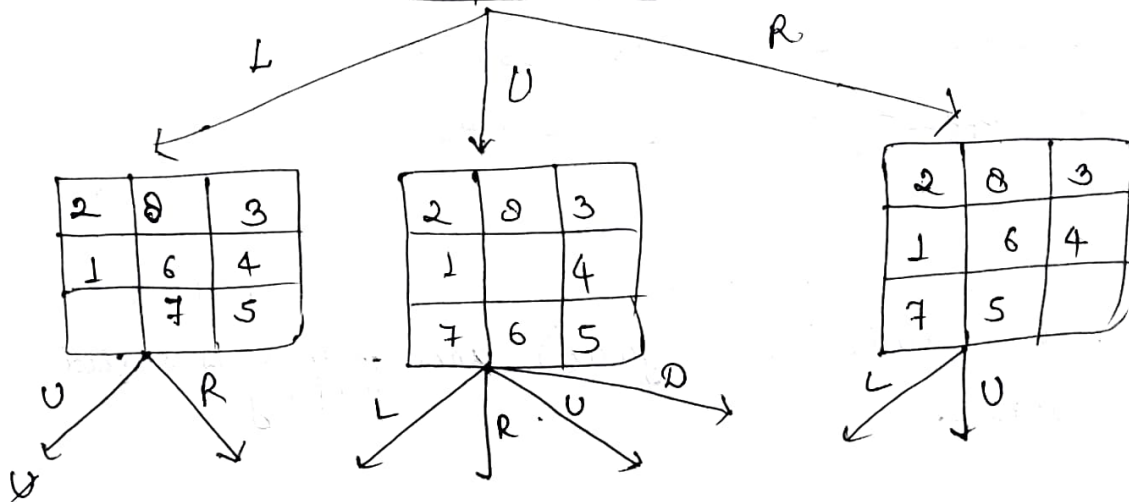
1	2	3
8		4
7	6	5

Goal State

Action Space = {L, R, U, D}

2	8	3
1	6	4
7		5

HW → Write a code to solve the puzzle and try to make it intelligent



Uninformed Search Algorithm -

aka blind search algorithms

↳ in such searches, the information is not available therefore agent has to go for brute force technique or blind search.

Things to be discussed in this category - problem is formulated as

- 1) BFS $\rightarrow$ queue
- 2) DFS $\rightarrow$ stack
- 3) DLS (Depth limited search) $\rightarrow$ stack
- 4) Uniform Cost Search $\rightarrow$ priority queue

} state space search tree

max no. of child of the node

Ques) Suppose the time & memory required for BFS with a branching factor = 10. Assume that 1 million nodes can be generated per second and one node requires 1000 bytes of storage. Find the solution at depth = 2, 4, 6.

Ans) depth = 0 to $n-1$ (n = no. of levels)
 $b = 10$
 $d=0 \rightarrow$ no. of nodes = 1
 $d=1 \rightarrow$ no. of nodes = 10
no. of nodes at $d=2 \rightarrow$ worst case = 10^2

$$\text{till } d=2 \text{ no. of nodes} = 111 \approx 10^2 = 10^2 + 10^1 + 10^0$$

$$\text{till } d=4 \text{ no. of nodes} = 11111 \approx 10^4 = 10^4 + 10^3 + 10^2 + 10^1 + 10^0$$

$$\text{till } d=6 \text{ no. of nodes} = 111111 \approx 10^6 = 10^6 + 10^5 + 10^4 + 10^3 + 10^2 + 10^1 + 10^0$$

$$\text{memory req at each step} = d=2 \rightarrow 10^5$$

$$d=4 \rightarrow 10^7$$

$$d=6 \rightarrow 10^9$$

$$\text{time req at each step} = d=2 \rightarrow 10^{-4}$$

$$d=4 \rightarrow 10^{-2}$$

$$d=6 \rightarrow 1 \text{ sec}$$

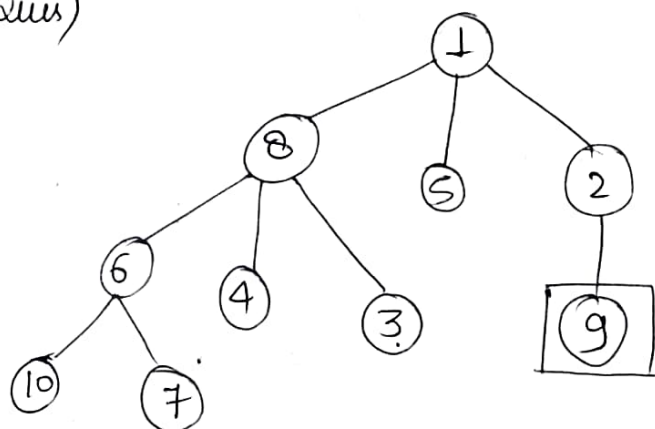
time complexity = $(b)^d$ $\rightarrow$ in worst case

$d \rightarrow$ depth at which you get the solution.

space complexity - we need to store the data of all the nodes there (in worst case)
 $O(b^d)$

* we will traverse the whole tree, if we want optimal solution. otherwise in level-order traversal we will get the solution of above problem quickly.

Ques)



Ans) $O(b \times d)$
 Space Complexity = ~~height of tree~~ = ~~$O(d)$~~ $O(d)$
 Time Complexity = $O(b^d)$

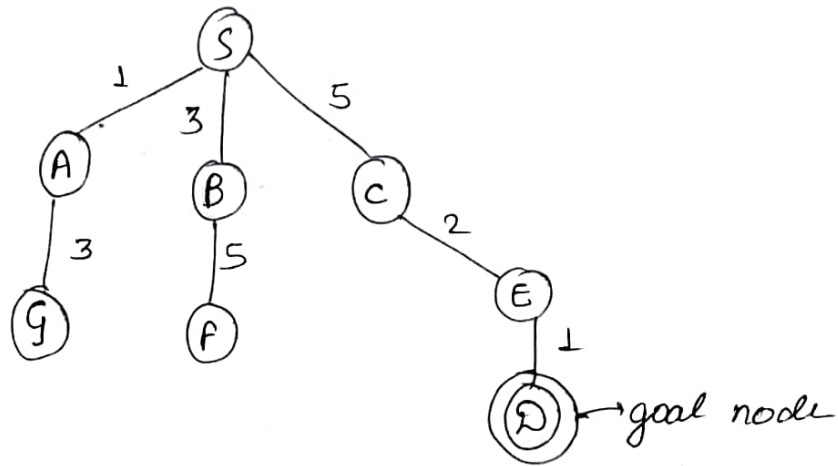
- ↳ if solution is present at depth then dfs is better.
- ↳ if solution is at upper levels use bfs
- ↳ generally dfs gives more optimal solution.

↳ Depth Limited Search -

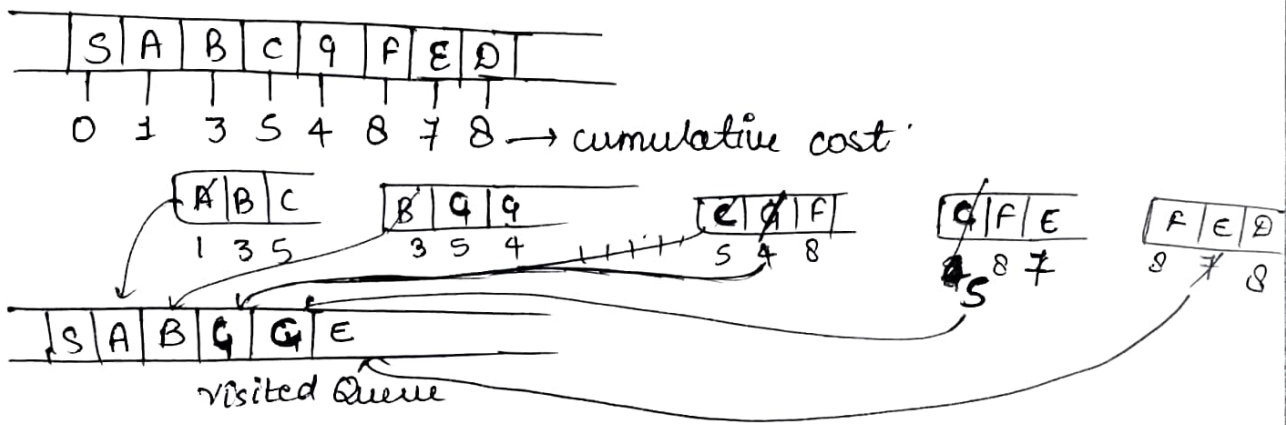
- ↳ used when few branches of tree are exceptionally long
- ↳ here we put a limit 'l' in depth → here we backtrack if we didn't get solution till.
- $$\left. \begin{array}{l} TC = O(b^l) \\ SC = O(b \times l) \end{array} \right\} \text{worst case.}$$

→ problem → we get no solution if solution goes beyond 'l'

Uniform Cost Search -



Ans) Priority Queue - high priority on low cost.



b = branching factor

d = best case depth of a solution

c = cost of optimal solution

ϵ = min cost of any action.

$$TC = O(b^d)$$

$$SC = O(b^d)$$

$$\text{Cost} = O(b^{(1 + \lceil c/\epsilon \rceil)})$$

total no. of actions

taken to reach till solution

↳ worst case

$$= \lceil c/\epsilon \rceil + 1$$

Informed Search Algorithm

↳ they possess a heuristic approach on how problem will be solved.
evaluation function.

algorithms involved in informed based search -

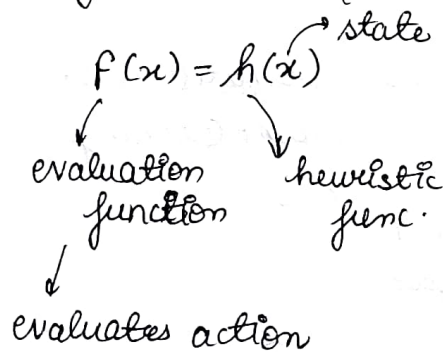
→ Greedy best first search (extension of uniform cost search)

→ A* Algorithm (better than above algorithm)

→ AO* Algorithm

→ Hill climbing

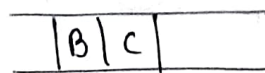
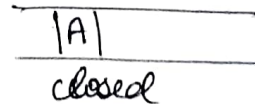
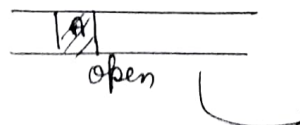
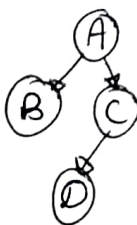
① Greedy Best First Search -



$$= h(A)$$

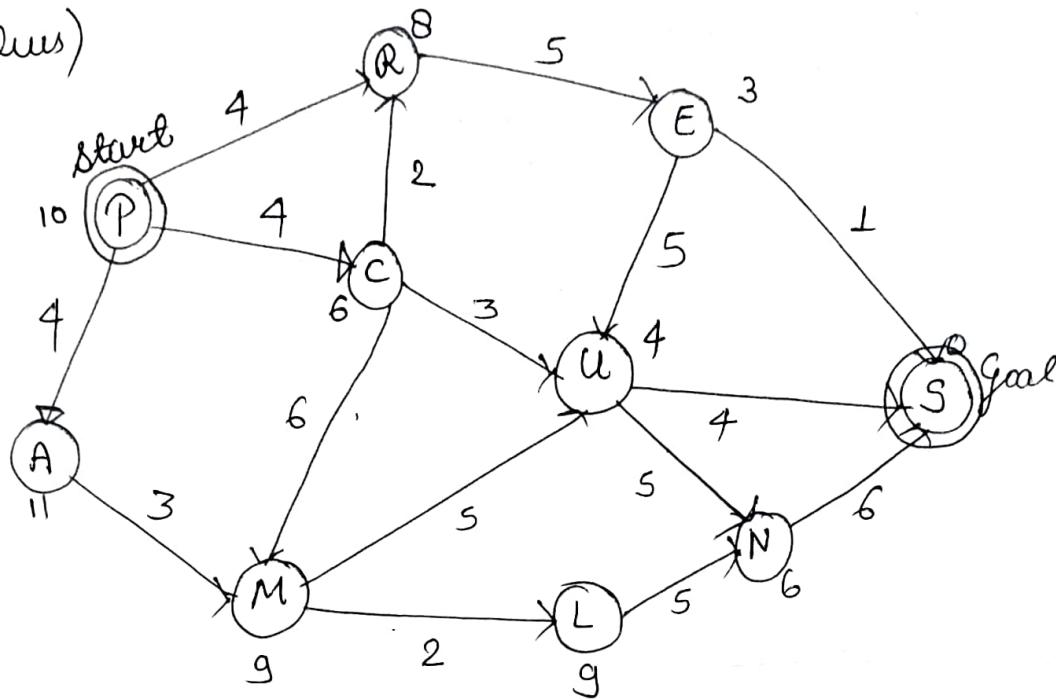
→ estimated distance of a from goal.

- in order to search in any problem we have to explore it. Therefore, two lists are created in which one is an open list and another one is closed list
- open list stores the nodes that we want to explore or should be explored (exploration = possible processes / nodes associated with the current node)
- closed list holds the visited nodes or explored nodes.



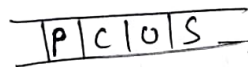
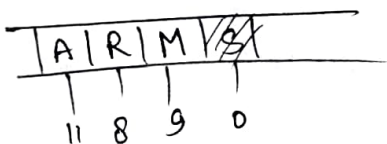
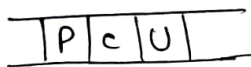
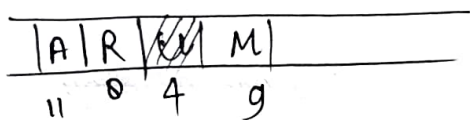
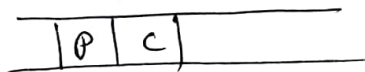
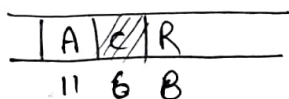
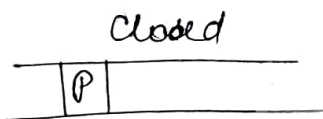
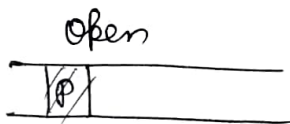
↳ based on $h(x)$

Ques)



edges = cost, node = heuristic value (estimated distance from reaching the goal)
 ↓
 It can always be overestimated or underestimated

Solun)



↪ path length = 11

↪ problem of gbf - does not provide the optimal solution.

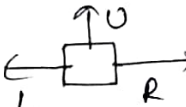
Ques) heuristic $\rightarrow$ total no. of misplaced tiles.
($h(x)$)

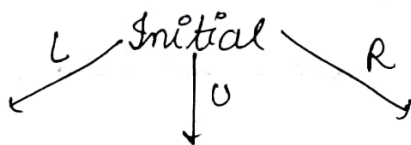
2	8	3
1	6	4
7		5

Initial

1	2	3
8		4
7	6	5

Goal.

Ans) possible movements $\Rightarrow$ 



2	8	3
1	6	4
	7	5

2	8	3
1		4
7	6	5

2	8	3
1	6	4
7	5	

(no. of misplaced tiles = 6)

$\hookrightarrow$ Including blank

(no. of misplaced tiles = 3)

(taking this)

(no. of misplaced tiles = 7)

$\hookrightarrow$ including blank tiles



2	8	3
	1	4
7	6	5

(4)

2	8	3
1	4	
7	6	5

(5)

2		3
1	8	4
7	6	5

(4)

2	8	3
1	6	4
7		5

(5)



	2	3
1	8	4
7	6	5

2	8	3
1		4
7	6	5

2	3	
1	8	4
7	6	5

② A* Algorithm-

→ here heuristic function's value and costs together make the evaluation function. $f(x) = g(x) + h(x)$

estimated value (estimated value from n^{th} node to goal node)

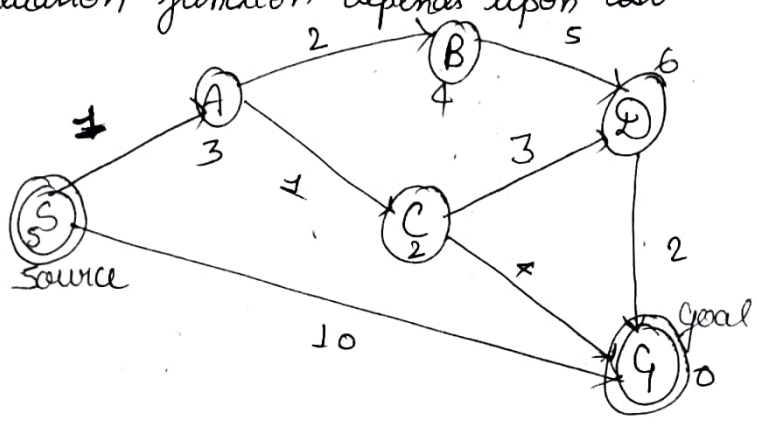
evaluation function

distance of x from initial state to state x (cost)

distance of x from goal.

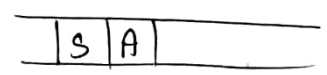
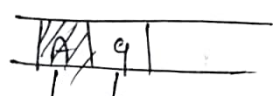
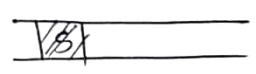
→ evaluation function depends upon cost & heuristic function.

Ques)

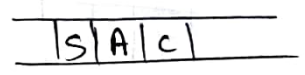
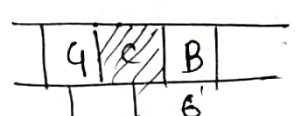


It provides ^{optimal} solution in case of admissibility.

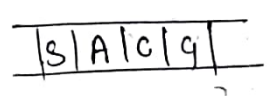
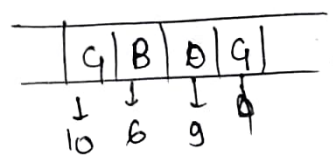
Ans)



$1+3=4$ $10+0$



$10+0$ 3



goal reached

→ path length = 6

Optimal solen depends on choice of heu. func.

this might change in case of over or under estimation in any node

⑧ optimal solution = solution with eco properties

- exhaustiveness (must explore all the search spaces)
- completeness (if solution exist then it must be gotten)
- optimality (solution must be best among all possible solen)

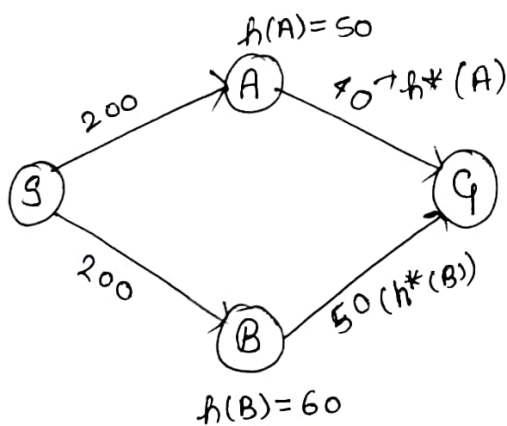
→ over estimation reduces the ~~prope~~ probability of getting an optimal solution.

heuristic func. can be either ($h(x)$)

- Exact Estimation $h(x) = h^*(x)$
- Over Estimation $h(x) > h^*(x)$
- Under Estimation. $h(x) < h^*(x)$

$h^*(x)$ = original cost

Case 2: Overestimation -



$$\cancel{f(A) = 240} \quad f(S \rightarrow A) = g(A) + h(A)$$

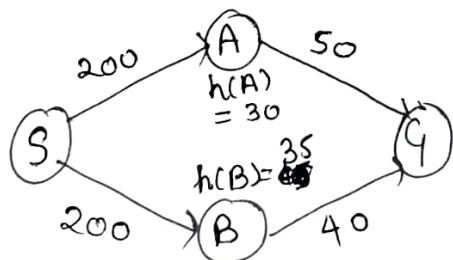
$$\checkmark \quad f(A) = 250$$

$$f(S \rightarrow B) = g(B) + h(B)$$

$$f(B) = 260$$

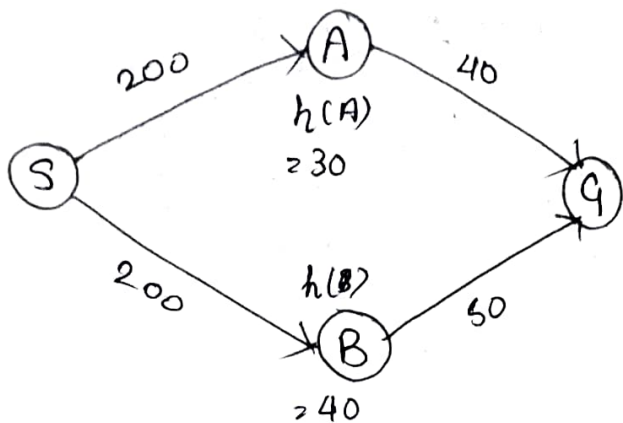
just in case, $h^*(B) = 35$ searching would have been stopped after $f(A)$ where as optimal solen would be $f(B)$

Case 3: Underestimation -



$$230 < 250$$

$$235 < 240$$



$240 > 230$ ✓

$250 > 240$
 ~~$240 < 250$~~

↳ possibility of exploration is always higher than in underestimation.

* Property of admissibility -

choosing a heuristic such that exhaustiveness and optimality of algorithm increase.

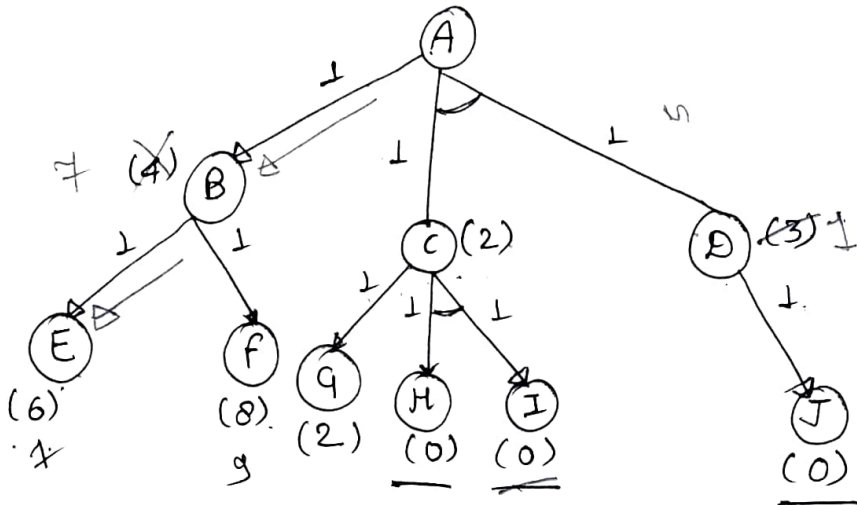
* this algorithm is used in -

③ AO* Algorithm -

$$f(n) = g(n) + h(n)$$

evaluation func.

→ there is a chance of updation in heuristic with exploration.



$$f(A \rightarrow B) = 4 + 1 = 5 \checkmark$$

$$f(A \rightarrow CD) = 1 + 1 + 2 + 3 = 7$$

$$\cancel{f(B \rightarrow A)} \quad f(B \rightarrow E) = 7 \checkmark, \quad f(B \rightarrow F) = 9$$

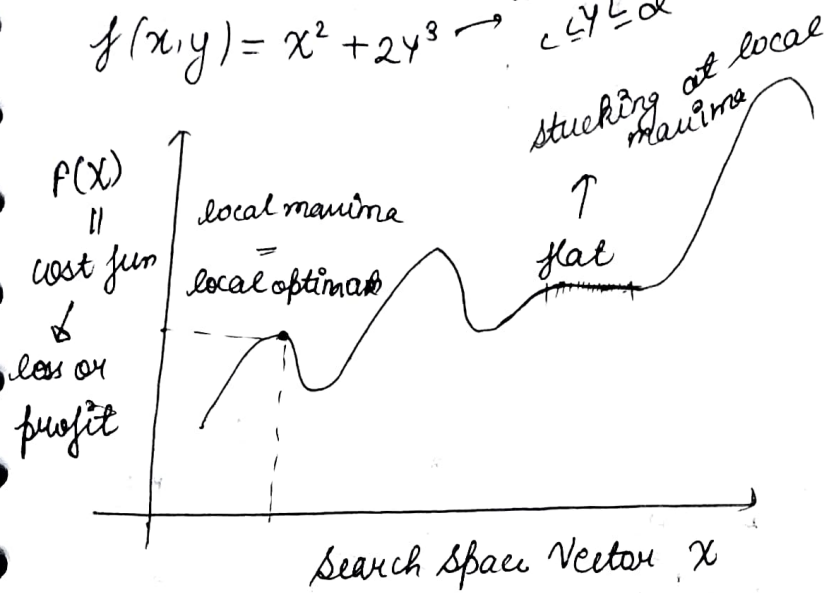
$f(B \rightarrow B \rightarrow E) \rightarrow$ not optimal solution $\rightarrow$ backtrack and upgrade the heuristic.

upgradation $\hat{=}$ min of $B \rightarrow E$, --, $B \rightarrow F = 7$

$$f(A \rightarrow CD) =$$

④ Hill Climbing $\rightarrow$ ^{Right Knight}

$$f(x, y) = x^2 + 2y^3 \rightarrow \begin{matrix} a \leq x \leq b \\ c \leq y \leq d \end{matrix}$$



here the heuristic is $\rightarrow$ how much problem have you already solved.

This algorithm revolves around the approach -

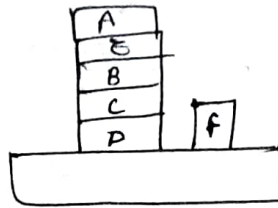
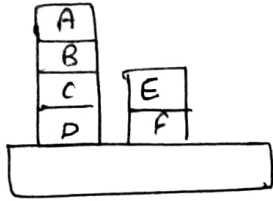
$\rightarrow$ begins with an initial solution that is chosen at random (most probably)

$\rightarrow$ identify neighbouring states of current solution by making small adjustments.

$\rightarrow$ if one of the neighbouring states offers a better solution than the current solution, then move to next state.
 $\rightarrow$ this process continues until you reach the local minima.

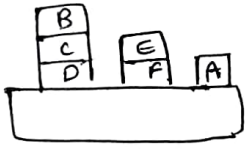
$\rightarrow$ local minima is also the local optima.

Qus) Objective is to maximize points - rightly placed = +1
 wrongly placed = -1
 Goal = -1



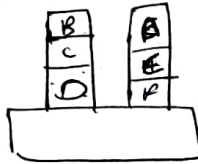
A → ground
 top of E
 E → A
 ground

P



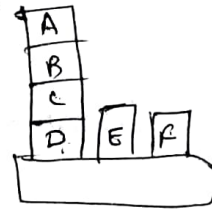
(2)

Q



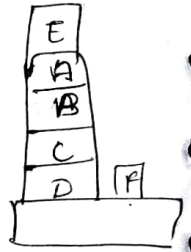
(4)

R



(2)

S

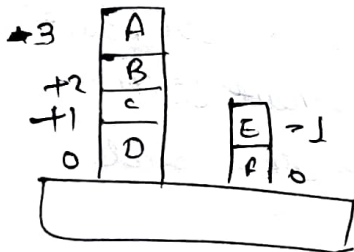


(2)

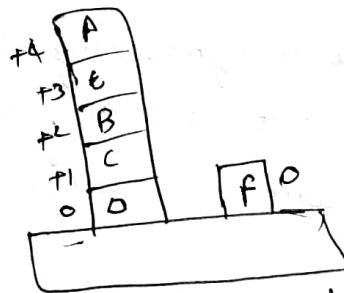
✓ stuck at local ~~minima~~ optima

→ there by adding move value to heuristic, observing the environment after the state.

updated heuristic → award if tiles are rightly placed below it
 punishment if wrongly placed.



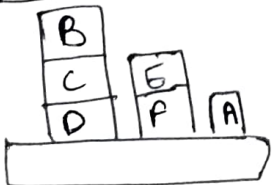
cost = -1



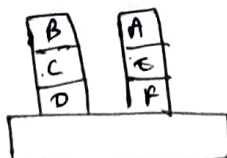
cost = +10

updated costs-

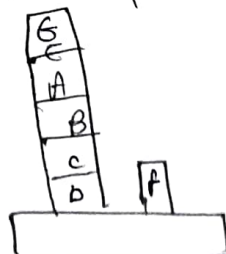
P = 2



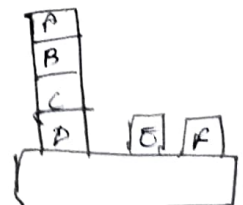
Q = 0



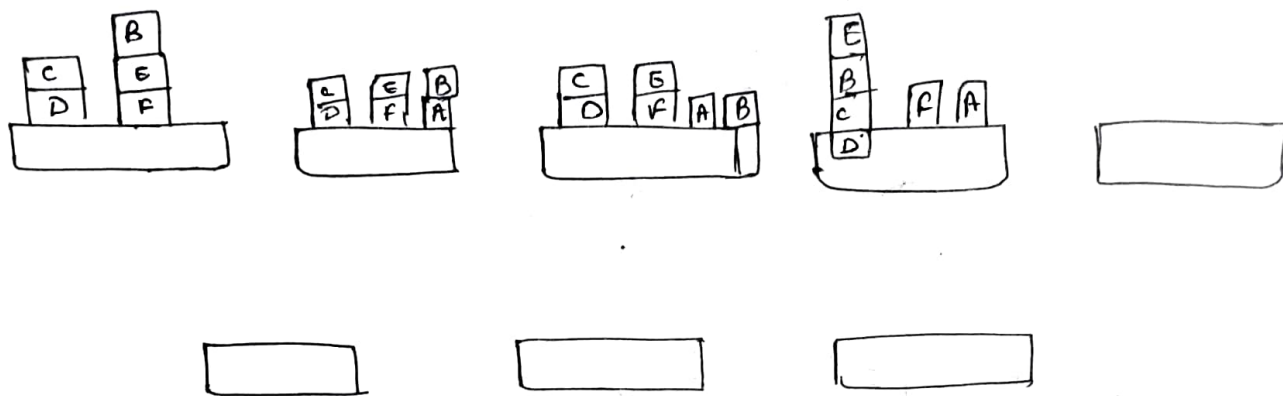
R = -4



S = 0



possible states -



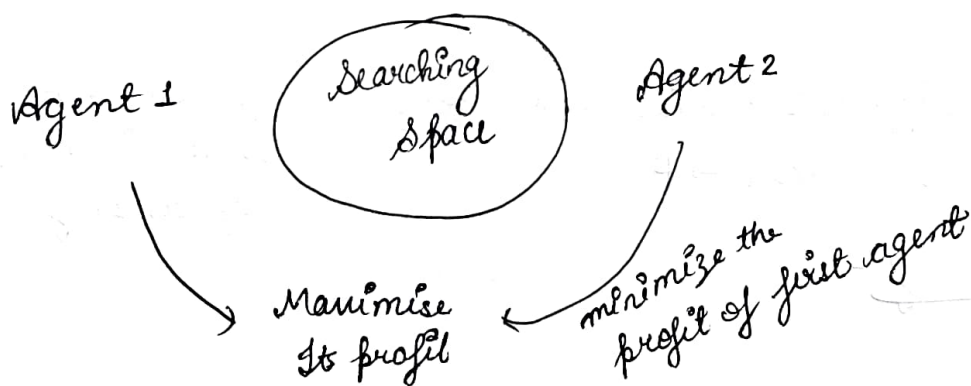
↳ ~~globally~~ fully observable environment, global info.

↳ algorithm is complete if heuristic is global heuristic

Adversarial Search Algorithms - (eg - GAN)

(31-Jan-25)

↳ here we talk about two agents, who compete with each other and their goals vary however search space is same for both the agents.

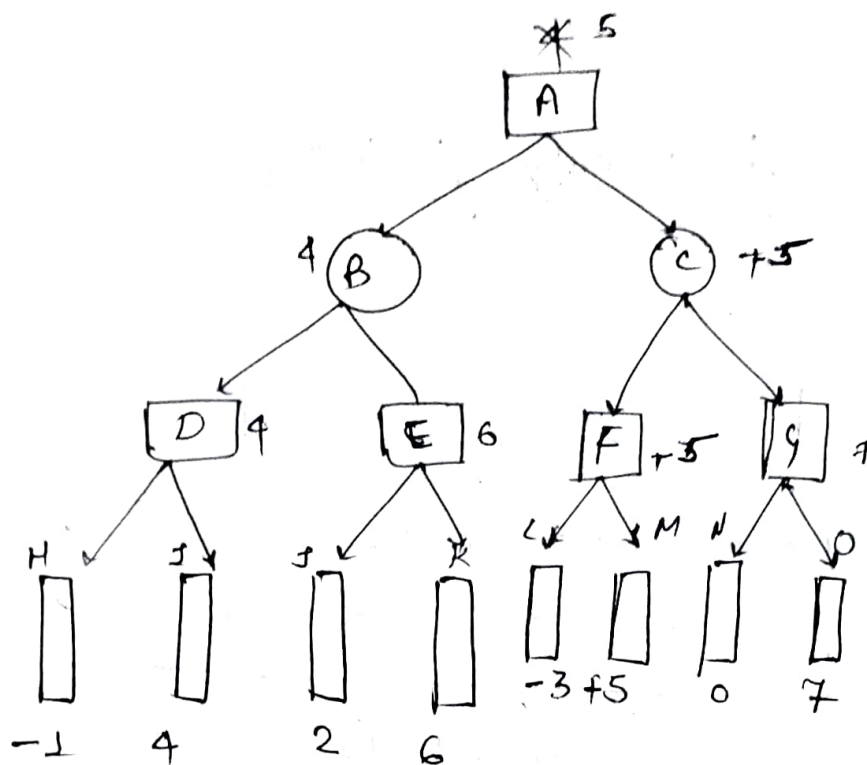


Algorithm used → MAX-MIN Algo.

□ → MAX

○ → MIN

▮ → Terminal



traversing the tree using dfs.

$$TC = b^d$$

$$SC = bd$$

↳ assigning different variables to both the agents, that will be modified by the agents only.
 max → α min → β

↳ α - β pruning -

Probabilistic Reasoning

7/2/25

- Bayes Theorem and Naïve Bayes based classifiers.
- Bayesian statistics and Bayes network

① Bayes Theorem -

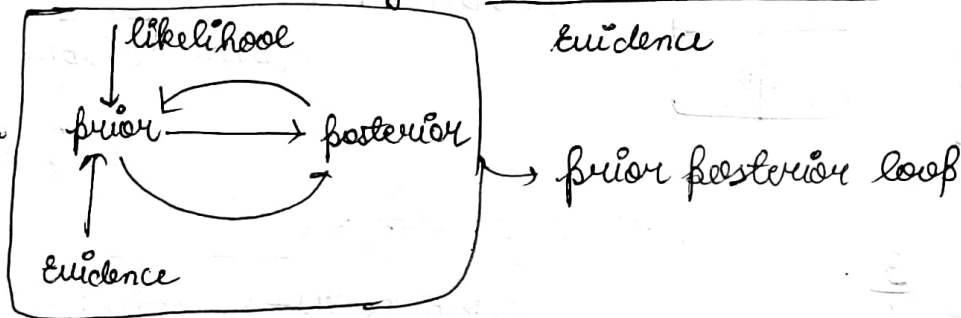
$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

posterior probability
of a particular event.

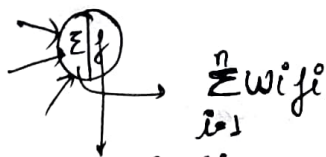
$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$

posterior probability = likelihood x Prior



- what is a model? set of parameters.
- total no. of input neurons in a neural network is the no. of columns in CSV / test file. and other layers are hidden layers or abstract layers.
- neurons are basically the computational units of a model that holds the summation of all input values with their randomly assigned weights.



activation function } used to add non-linearity
(RELU, LeakyRELU, tanh)

$$P(\text{hypothesis} | \text{observation}) = \frac{P(\text{observation} / \text{hypothesis}) \times P(\text{hypothesis})}{P(\text{observation})}$$

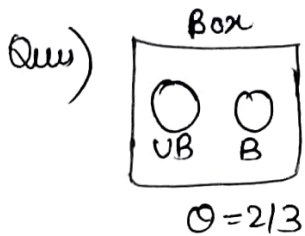
θ theta = collection of parameters

$$P(\theta | \text{data}) = \frac{P(\text{data} / \theta) \times P(\theta)}{P(\text{data})}$$

model is rep. by theta (θ)

Frequency view of probability is used in neural networks because it helps us to approximate the value of function. It is based on Bayes' theorem.

This approach fails when no. of experiments is less.



B	
$1/2 H$	$1/3 T$
UB	
$1/2 H$	$1/2 T$

$$P(UB/H) = \frac{P(UB \cap H)}{P(B \cap H) + P(UB \cap H)}$$

total prob.



$$\frac{3}{7}$$

$$P(UB/H) = \frac{P(H|UB) * P(H)}{P(H|B) * P(H) + P(H|UB) * P(H)}$$

for classification models, there is no significance of total probability.

Naive Bayes' assumption -

Two vectors -

① Feature Vector = $\{x_1, x_2, x_3, \dots, x_n\}$

Target class Vector = $\{c_1, c_2, \dots, c_k\}$
 $\hookrightarrow K$ classes

$X = \text{test data}$, $P(C_1/X)$, $P(C_2/X)$, $P(C_3/X)$ $\dots$, $P(C_n/X)$
probability vector of class data

$$P(C_k/X) = P(C_k) \cdot \prod_{i=1}^n P(x_i/C_k)$$

$$= \frac{1}{Z} P(C_k) \prod_{i=1}^n P(x_i/C_k)$$

$\underbrace{\hspace{1cm}}_{\text{some constant}}$

$$= P(C_k) \prod_{i=1}^n P(x_i/C_k)$$

$$\rightarrow y_p = \arg \max_{K \in \{1, 2, 3, \dots, K\}}$$